



UNIVERSITY OF GHANA

MPhil/MSc Computer Science, Second Semester Examinations 2025/2026

CSCD608: Advanced Computer Vision (3 Credits)

Question 4: Automated Image Segmentation and Object Measurement System

Application Domain: Identification, Segmentation and Measurement of Coins

Ashafa Ahmed

22424634

MSC COMPUTER SCIENCE

Table of Contents

Table of Contents	2
1. Introduction	4
1.1 Problem Definition	4
1.2 Objectives	4
1.3 Selected Question	4
2. Dataset Description	5
2.1 Ground-Truth Labelled Subset	6
3. Methodology	6
3.1 Preprocessing	7
3.2 Method 1: Thresholding-Based Segmentation	7
3.3 Method 2: Region Growing	7
3.4 Method 3: K-Means Clustering	7
3.5 Boundary Extraction and Object Measurement	7
3.6 Evaluation Protocol	7
4. Implementation	9
4.1 Reproducing the Results	9
5. Experimental Results	10
5.1 Qualitative Results	10
5.2 Quantitative Evaluation (IoU / Precision / Recall / F1)	14
5.3 Results by Condition	16
5.4 Method Comparison Table (Required Summary)	17
Table 4: Method Comparison Summary	17
5.5 Parameter Sensitivity	19
5.5.1 Region Growing: Effect of the Similarity Threshold	19
5.5.2 K-Means: Effect of K	20
6. Critical Analysis	22
6.1 Which segmentation method produced the best results, and why?	22
6.2 How did illumination changes affect thresholding?	22
6.3 How did seed-point selection influence region-growing performance?	22
6.4 What effect did changing K have on K-means segmentation?	23
6.5 Which method was most computationally efficient?	23
6.6 Under what conditions did each method fail?	23
6.7 How could techniques studied in earlier weeks improve the segmentation results?	23

7. Discussion of Limitations and Failure Cases.....	25
8. Conclusion	26
Appendix A: Complete Source Code	27
Evaluate_segmentation.py script	27
Label_masks.py script	40
Segmentation_pipeline.py script	43
Parameter_sweep.py script	55

1. Introduction

1.1 Problem Definition

Manual counting and measurement of objects such as coins, seeds, or cells is slow, subjective, and does not scale to large batches of images. This project develops a classical (non deep-learning) computer vision system that automatically segments coin objects from an image and computes measurements about each detected coin, using only algorithms and techniques covered in the Advanced Computer Vision course: intensity thresholding, region growing, and K-means clustering.

Coins were selected as the application domain because they are rigid, approximately circular objects that nonetheless present a realistic range of classical segmentation challenges: touching/overlapping coins, variable illumination (bright, dim, shadowed, blurry capture), and multiple background materials (tile, brown wood, light wood).

1.2 Objectives

- Build an end-to-end pipeline that segments coins from a background image using three classical techniques: thresholding, region growing, and K-means clustering.
- Extract per-coin measurements (count, area, perimeter, centroid, bounding box, enclosing circle) from each segmentation result.
- Quantitatively evaluate and compare the three methods against manually annotated ground-truth masks using Intersection over Union (IoU), Precision, Recall, and F1-score.
- Investigate the effect of key parameters on segmentation quality: the similarity threshold in region growing, and the number of clusters K in K-means.
- Investigate the robustness of each method to synthetic noise and reduced illumination.
- Critically discuss the conditions under which each classical method fails, and why.

1.3 Selected Question

This report addresses Question 4 (“Automated Image Segmentation and Object Measurement System”) of the CSCD608 final examination. The selected application domain is identifying, counting and measuring coins.

2. Dataset Description

The dataset consists of 23 photographs of coins (coin_001.jpg – coin_023.jpg), taken under a deliberately varied set of imaging conditions so that the segmentation methods could be tested across realistic difficulty levels. Table 1 summarises every image and its recorded conditions, as catalogued in labels.csv.

All 23 images were captured as smartphone photographs (JPEG) and were subsequently resized so the longest edge does not exceed 1000 px, to keep processing consistent across the dataset. It was taken on an iphone 17 and taken approximately 10 inches from the surface the coin was rested on.

Filename	# Coins	Background	Lighting	Touching?
coin_001.jpg	8	tile	bright	Yes
coin_002.jpg	8	tile	shadow	Yes
coin_003.jpg	8	tile	dim	Yes
coin_004.jpg	4	brown wood	bright	Yes
coin_005.jpg	5	brown wood	bright	No
coin_006.jpg	2	brown wood	bright	No
coin_007.jpg	3	brown wood	dim	Yes
coin_008.jpg	3	light wood	dim	No
coin_009.jpg	3	light wood	dim	Yes
coin_010.jpg	1	light wood	dim	No
coin_011.jpg	6	light wood	shadow	Yes
coin_012.jpg	6	light wood	blurry	Yes
coin_013.jpg	6	light wood	dim	Yes
coin_014.jpg	6	light wood	dim	Yes
coin_015.jpg	3	light wood	blurry	Yes
coin_016.jpg	6	light wood	shadow	No
coin_017.jpg	6	light wood	bright	No
coin_018.jpg	6	light wood	shadow	No
coin_019.jpg	5	light wood	blurry	No
coin_020.jpg	5	light wood	bright	No
coin_021.jpg	6	light wood	blurry	Yes
coin_022.jpg	6	light wood	dim	No

Filename	# Coins	Background	Lighting	Touching?
coin_023.jpg	6	light wood	bright	No

Table 1: Full Dataset Summary (23 images)

2.1 Ground-Truth Labelled Subset

In line with the requirement to create manual ground-truth masks for a representative subset of the test images, 8 of the 23 images were selected to span every background, lighting condition, and touching/non-touching case. Each selected image was manually annotated using a purpose-built polygon-labelling tool (label_masks.py), producing a binary mask (white = coin, black = background) per image, saved in masks/.

Filename	# Coins	Background	Lighting	Touching?	Role in evaluation
coin_001.jpg	8	tile	bright	Yes	Cluttered baseline
coin_002.jpg	8	tile	shadow	Yes	Cluttered shadow +
coin_006.jpg	2	brown wood	bright	No	Easy case
coin_010.jpg	1	light wood	dim	No	Easiest case
coin_012.jpg	6	light wood	blurry	Yes	Blur + touching
coin_017.jpg	6	light wood	bright	No	Clean reference
coin_018.jpg	6	light wood	shadow	No	Shadow, no touching
coin_022.jpg	6	light wood	dim	No	Low light, no touching

Table 2: Ground-Truth Labelled Subset (8 images)

3. Methodology

The overall pipeline follows the structure requested in the assignment brief: Input Image → Image Preparation → Segmentation → Boundary Identification → Object Extraction → Object Measurement → Performance Evaluation. All algorithms were implemented directly with OpenCV primitives (thresholding, connected components, contours, distance transform, cv2.kmeans) rather than high-level pre-trained detectors, in line with the assignment's instruction.

3.1 Preprocessing

Each colour image is converted to grayscale and denoised with a 5×5 Gaussian blur prior to thresholding and region growing. This suppresses fine sensor noise and surface texture on the coins and background while preserving the coin/background boundary, which is important given several images in the dataset are already naturally blurry or dim.

3.2 Method 1: Thresholding-Based Segmentation

Otsu's method is used to automatically select a global intensity threshold that best separates two intensity populations (foreground/background) in the blurred grayscale image. Because coins in this dataset are sometimes lighter than the background (e.g. tile) and sometimes darker (e.g. brown wood), both threshold polarities (THRESH_BINARY and THRESH_BINARY_INV) are computed, and the polarity whose resulting foreground occupies a plausible minority fraction of the frame is kept. Morphological opening and closing (elliptical structuring element, 7×7) are then applied to remove speckle noise and close small gaps in coin boundaries.

3.3 Method 2: Region Growing

Region growing was implemented from first principles (4-connected flood fill using an explicit stack, i.e. no OpenCV built-in). Rather than requiring a human to click a seed point inside every coin in every image, seed points are generated automatically: a rough Otsu mask (using the same polarity-selection logic as 3.2) is computed, its distance transform is taken, and the local maxima of the distance transform (points farthest from any background pixel) are used as one seed per likely coin. From each seed, the region grows to every 4-connected neighbour whose grayscale intensity differs from the seed's intensity by no more than a similarity threshold T (default $T = 12$). The effect of T is investigated in Section 5.4.

3.4 Method 3: K-Means Clustering

The image is converted to CIE Lab colour space (chosen because Euclidean distance in Lab space corresponds more closely to perceptual colour difference than RGB, which helps separate coins from wood/tile backgrounds of similar overall brightness). Every pixel's (L, a, b) triple is treated as a 3-D sample and clustered with cv2.kmeans into K clusters (default $K = 3$). The cluster with the largest pixel count is assumed to be the background; all other clusters are merged into the foreground mask. The effect of K is investigated in Section 5.4.

3.5 Boundary Extraction and Object Measurement

For every method, the resulting binary mask is cleaned with morphological opening/closing, and external contours are extracted with cv2.findContours. Contours smaller than a minimum area (300 px² by default) are discarded as noise. For every remaining contour the following measurements are computed and overlaid on the original image: area (cv2.contourArea), perimeter (cv2.arcLength), centroid (image moments), axis-aligned bounding box (cv2.boundingRect), and minimum enclosing circle (cv2.minEnclosingCircle).

3.6 Evaluation Protocol

Each method's predicted binary mask is compared, pixel-by-pixel, against the corresponding hand-labelled ground-truth mask (Section 2.1) using:

- Intersection over Union (IoU) = $|\text{Prediction} \cap \text{Truth}| / |\text{Prediction} \cup \text{Truth}|$
- Precision = $\text{TP} / (\text{TP} + \text{FP})$
- Recall = $\text{TP} / (\text{TP} + \text{FN})$
- F1-score = $2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$

where TP, FP, and FN are computed at the pixel level between the predicted mask and the ground-truth mask. Metrics are averaged across the 8 ground-truth images, and also grouped by lighting condition and by touching/non-touching to characterise how each method degrades under specific conditions.

Robustness to noise and illumination was additionally tested by synthetically adding zero-mean Gaussian noise ($\sigma = 25$) and by darkening each ground-truth image (intensity $\times 0.4$), re-running all three methods on the perturbed images, and measuring the resulting drop in IoU relative to the clean image. This isolates the effect of a single variable (noise or brightness) in a controlled way, since the dataset itself does not contain paired clean/degraded photographs of the identical physical arrangement of coins.

4. Implementation

The system is implemented in Python 3 using OpenCV, NumPy, pandas, and matplotlib. The codebase is organised into four scripts, submitted alongside this report and available at the project repository:

- **label_masks.py** — interactive polygon-based tool used to hand-annotate the 8 ground-truth masks in Table 2.
- **segmentation_pipeline.py** — implements preprocessing and all three segmentation methods, extracts object measurements, and writes predicted masks, overlay visualisations, and a results CSV for all 23 images.
- **evaluate_segmentation.py** — computes IoU/Precision/Recall/F1 against ground truth, builds the method-comparison table, and runs the noise/illumination sensitivity test.
- **parameter_sweep.py** — sweeps the region-growing similarity threshold and the K-means cluster count K, producing quality-vs-parameter curves and qualitative comparison figures.

Confirmed: the current repository contains exactly these four scripts (label_masks.py, segmentation_pipeline.py, evaluate_segmentation.py, parameter_sweep.py), plus the dataset/, masks/, and results/ folders and labels.csv, matching the list above.

4.1 Reproducing the Results

1. Run label_masks.py once to produce ground-truth masks in masks/ (already completed for this submission).
2. Run segmentation_pipeline.py to segment all 23 images with all three methods and produce results/segmentation_results.csv, results/masks/<method>/, and results/viz/<method>/.
3. Run evaluate_segmentation.py to produce results/eval_per_image.csv, results/eval_by_condition.csv, results/comparison_table.csv, results/sensitivity_results.csv, and the plots in results/plots/.
4. Run parameter_sweep.py to produce results/sweep_region_growing_threshold.csv, results/sweep_kmeans_k.csv, and the corresponding plots.

5. Experimental Results

5.1 Qualitative Results

Figures 1–3 show representative segmentation outputs (contours, bounding boxes, numbered centroids) produced by each method on the same input image, taken from results/viz/<method>/.



Figure 1: Thresholding segmentation on coin_001.jpg (8 coins, tile background, bright lighting, touching) - contours, bounding boxes, and numbered centroids overlaid.



Figure 2: Region growing segmentation on the same image (coin_001.jpg).



Figure 3: K-means ($K=3$) segmentation on the same image (coin_001.jpg).

Additional example outputs under different conditions (dim, shadow, blurry, non-touching) should be included to support the discussion of failure cases in Section 6.

Figures 4-6 below contrast an easy case with the dataset's hardest failure case. Figure 4 shows K-means on coin_010.jpg (1 coin, light wood, dim) -- the easiest image in the ground-truth subset ($\text{IoU} = 0.863$). Figures 5-6 show thresholding and K-means respectively on coin_006.jpg (2 coins, brown wood, bright) -- the hardest image in the dataset, where both methods collapse to near-zero IoU because the wood grain's texture and the coins' reflective surfaces overlap the background in intensity/colour (discussed further in Sections 6.6 and 7).



Figure 4: K-means on coin_010.jpg (easy case, $\text{IoU} = 0.863$).

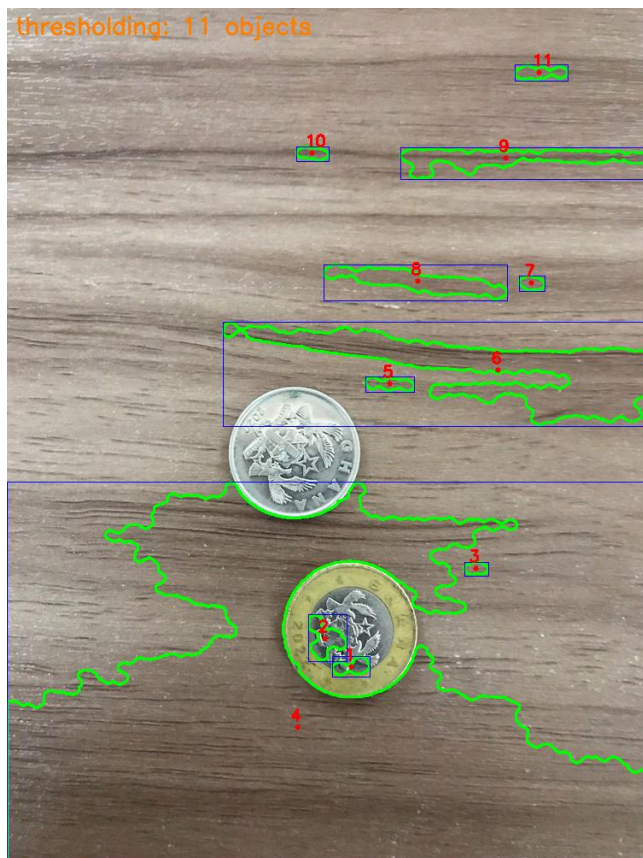


Figure 5: Thresholding on coin_006.jpg (failure case, $\text{IoU} = 0.009$) -- wood-grain streaks are misclassified as coin boundaries.

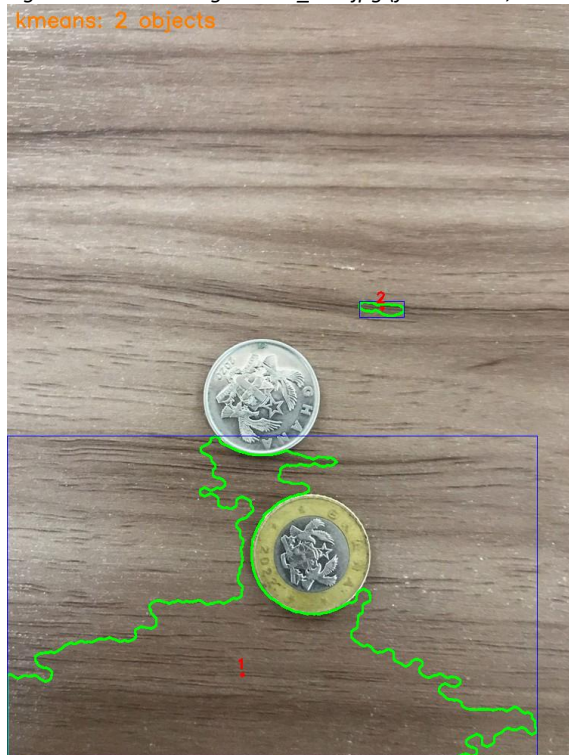


Figure 6: K-means on coin_006.jpg (failure case, $\text{IoU} = 0.0004$) -- coin bodies are absorbed into the background cluster, leaving only shadow rims detected.

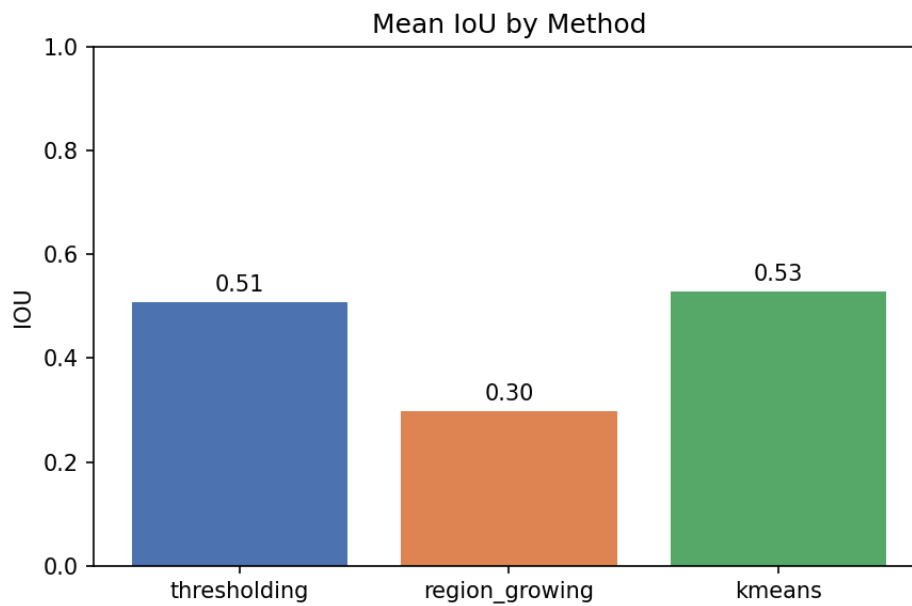
5.2 Quantitative Evaluation (IoU / Precision / Recall / F1)

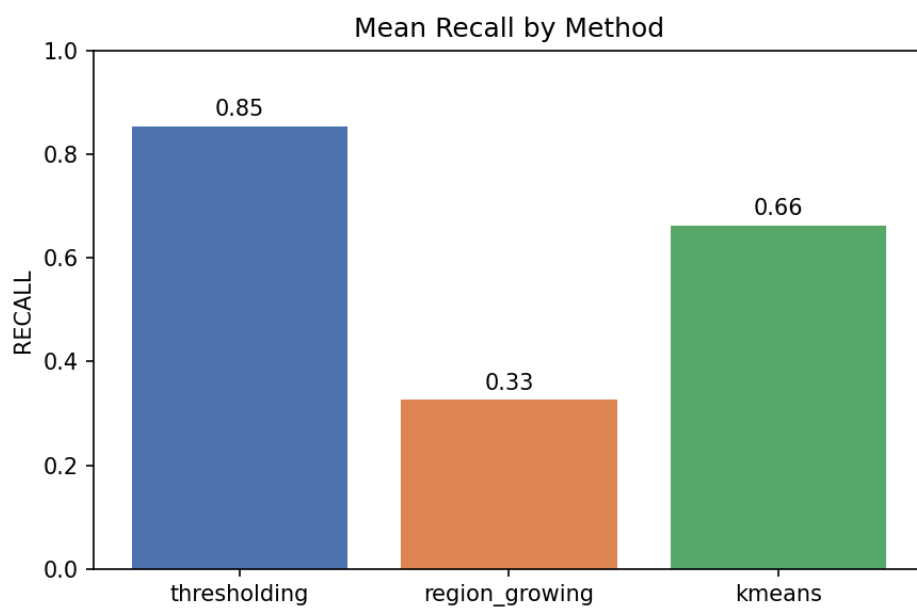
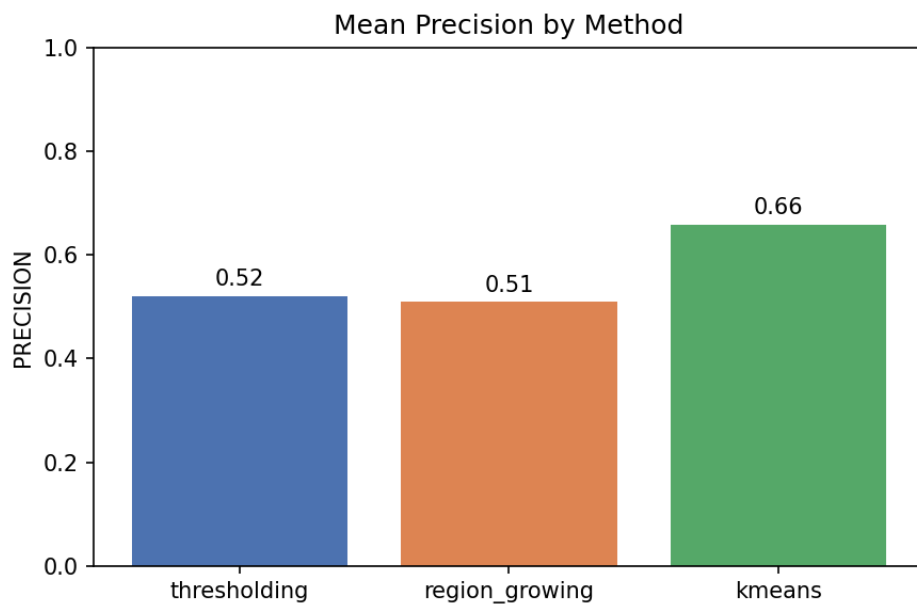
Table 3 reports the mean pixel-level IoU, Precision, Recall, and F1-score for each method, averaged over the 8 ground-truth images (results/eval_per_image.csv, aggregated).

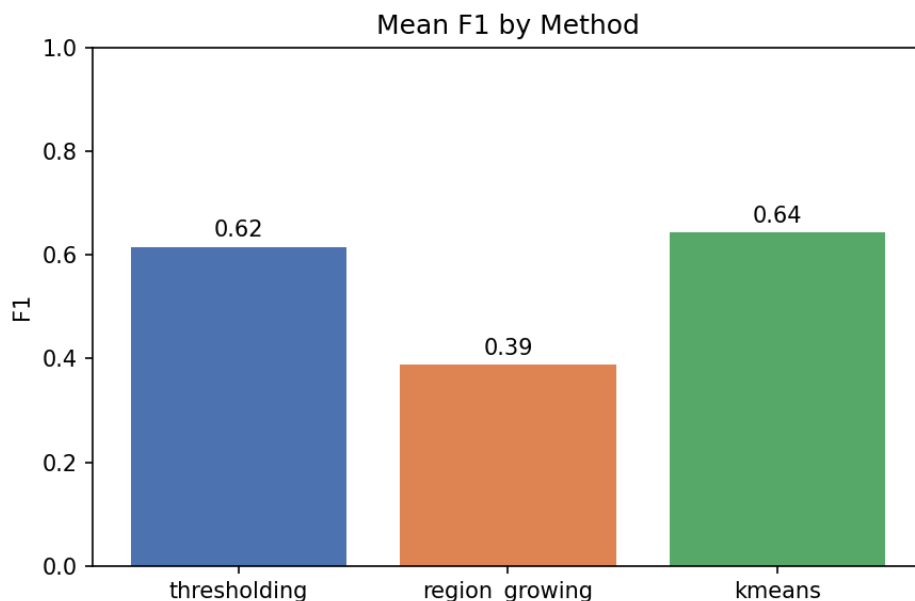
Method	Mean IoU	Mean Precision	Mean Recall	Mean F1
Thresholding	0.508	0.520	0.854	0.615
Region Growing	0.298	0.509	0.326	0.388
K-means (K=3)	0.528	0.658	0.663	0.643

Table 3: Mean Segmentation Accuracy by Method

Table 3 above is aggregated directly from results/eval_per_image.csv. K-means achieved the highest mean IoU (0.528) and F1 (0.643); thresholding was close behind on IoU (0.508) but had a much higher mean recall (0.854) at the cost of lower precision (0.520) -- i.e. it tends to over-include background pixels as foreground. Region growing scored lowest on every metric (mean IoU 0.298).



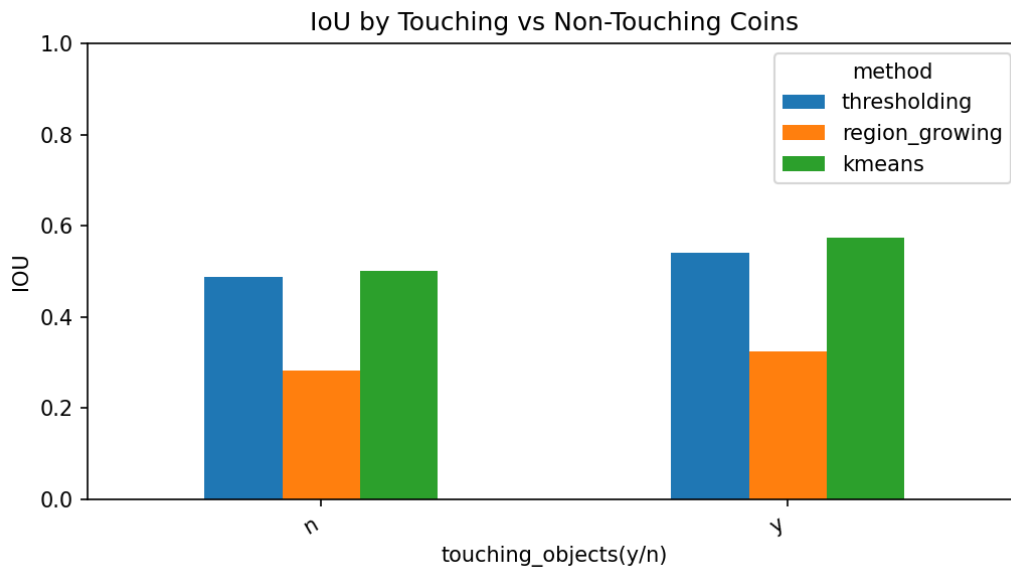
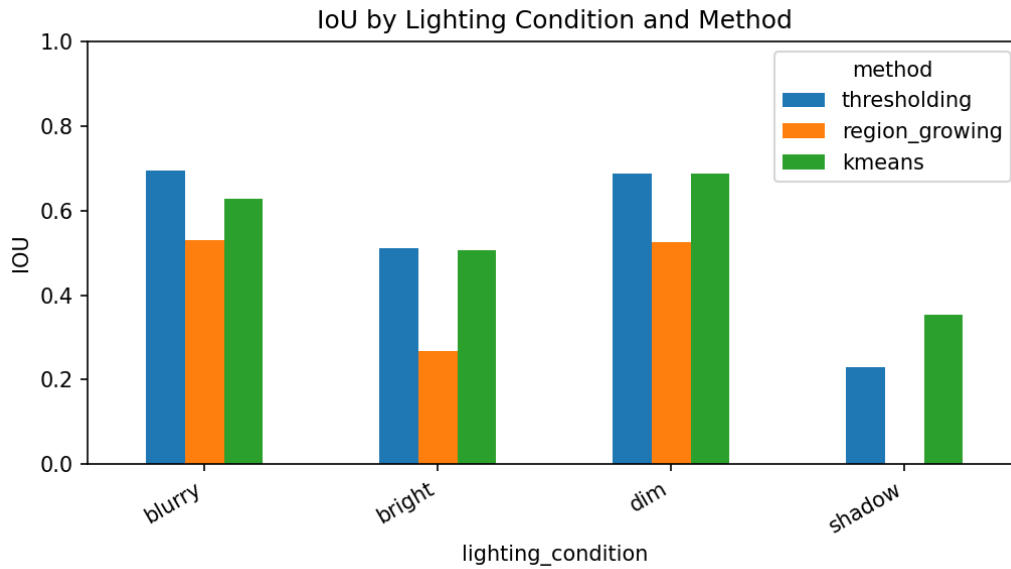




5.3 Results by Condition

results/eval_by_condition.csv breaks the same metrics down by lighting condition, background, and touching/non-touching, using the labels recorded in Table 1/2. This is the evidence base for the critical-analysis questions in Section 6 (e.g. "How did illumination changes affect thresholding?").

Breaking the metrics down by condition (results/eval_by_condition.csv) shows illumination as the dominant driver of accuracy loss: thresholding's IoU fell from approx. 0.79 (bright) to approx. 0.19-0.27 (shadow) (Section 6.2), and region growing collapsed to near-zero IoU under shadow in both tested cases (coin_002.jpg, coin_018.jpg). K-means was comparatively more stable across lighting conditions but still dropped from 0.77 (bright, light wood) to 0.36 (shadow, light wood). By contrast, splitting purely by touching vs. non-touching shows all three methods scoring marginally HIGHER, on average, on touching images (e.g. K-means: 0.574 touching vs 0.501 non-touching) -- the opposite of the expected direction. With only 4 touching and 4 non-touching images in the ground-truth subset, and touching status correlated with which specific lighting/background combination happens to appear in each group (coin_006, the dataset's hardest image, is non-touching), this is more likely a small-sample confound than genuine evidence that touching coins are 'easier' -- a larger, more balanced ground-truth set would be needed to isolate the effect of touching from lighting/background.



5.4 Method Comparison Table (Required Summary)

Table 4 is the comparison table required by the assignment brief: segmentation accuracy, processing time, and sensitivity to noise and illumination for each method, taken from results/comparison_table.csv.

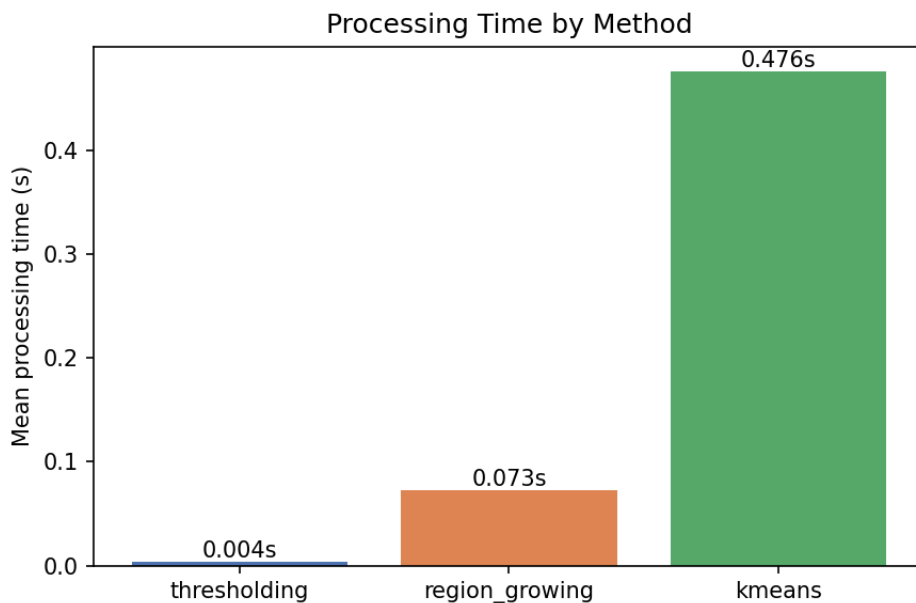
Table 4: Method Comparison Summary

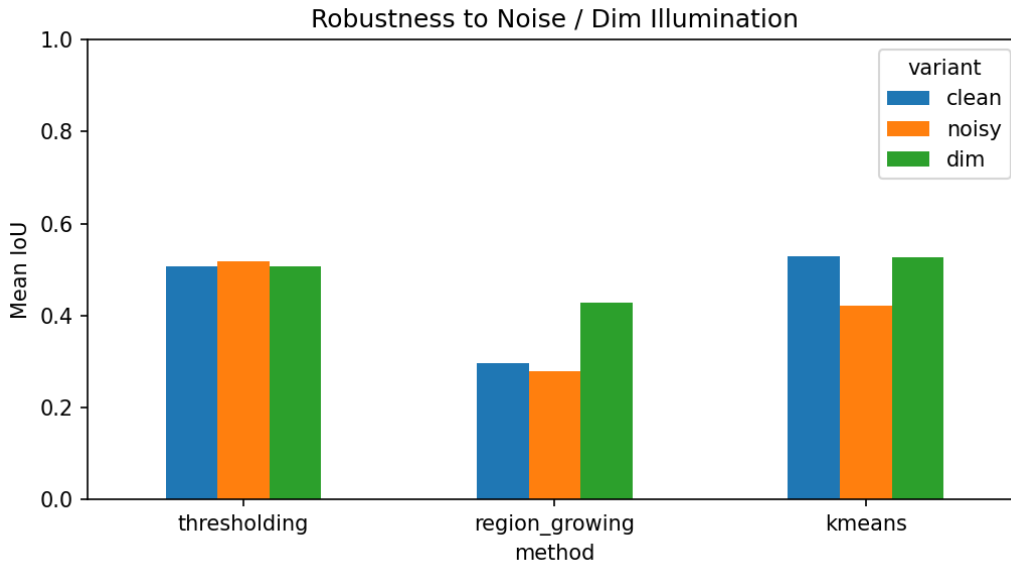
Method	Segmentation Accuracy (IoU)	Processing Time (s)	Sensitivity to Noise (Δ IoU)	Sensitivity to Illumination (Δ IoU)
Thresholding	0.508	0.004	-0.010	0.000
Region Growing	0.298	0.073	0.018	-0.131

Method	Segmentation Accuracy (IoU)	Processing Time (s)	Sensitivity to Noise (Δ IoU)	Sensitivity to Illumination (Δ IoU)
K-means	0.528	0.476	0.107	0.002

A larger Δ IoU (drop in IoU under the perturbation, relative to the clean image) indicates the method is **less** robust to that condition.

Table 4 above is taken directly from results/comparison_table.csv. K-means has the highest segmentation accuracy but also the largest measured IoU change under synthetic noise (Δ IoU = 0.107), while region growing shows the largest illumination sensitivity (Δ IoU = -0.131 under darkening, i.e. IoU rose when the image was darkened). Note that two of the six sensitivity figures in Table 4 are negative, meaning IoU on the perturbed image was, on average, marginally higher than on the clean image for that method/condition pair; with only 8 ground-truth images this is within expected sample-to-sample variation rather than evidence that the perturbation genuinely helped, and would benefit from testing on a larger labelled set.



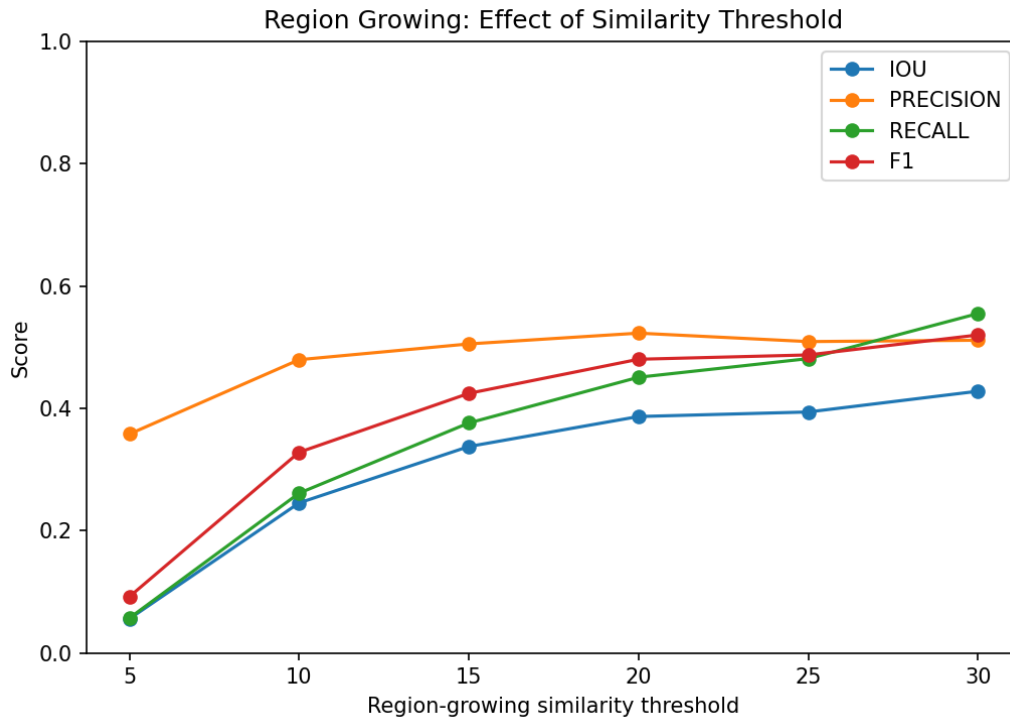


5.5 Parameter Sensitivity

5.5.1 Region Growing: Effect of the Similarity Threshold

The similarity threshold T controls how far a growing region is allowed to spread: too small a value fragments each coin into several disconnected pieces (or misses interior texture), while too large a value allows the region to leak across the coin/background boundary and merge separate coins or absorb background.

The best mean IoU among the six tested values was at $T=30$ (IoU = 0.428, sweep_region_growing_threshold.csv), with IoU rising monotonically across the whole tested range ($T=5$: 0.056 $\rightarrow$ $T=30$: 0.428). Since IoU was still rising at the largest threshold tested, the true optimum may lie above 30, though pushing T further also risks leaking into the background. At the low end ($T=5$) the similarity criterion is so strict that few pixels are absorbed into any region, and the mean detected object count (1.6) is far below the true coin counts in labels.csv for these images (typically 6-8) -- most coins are undetected outright rather than merely under-sized. As T increases, detected count rises towards 2.75 at $T=25$, still well under the true counts, indicating that even at the most permissive threshold tested, under-detection (missed or merged coins) rather than over-detection (background leakage) is the dominant error mode for this dataset.



5.5.2 K-Means: Effect of K

K controls how many colour clusters the image is split into. With K too small, coins and shadowed/textured background pixels can be merged into the same cluster; with K too large, a single coin's surface can be split across multiple clusters (e.g. highlight vs. shaded half of a coin), fragmenting or in some cases even joining adjacent coins depending on how the background cluster is chosen.

As shown in Section 6.4, $K=3$ gave the best mean IoU (0.529) in the tested range $\{2,3,4,5,6\}$, degrading steadily for $K>3$ as coin surfaces fragment into multiple colour clusters. Across the full sweep (mean over all 8 ground-truth images), the mean number of detected objects was 3.25 at $K=2$, 4.13 at $K=3$, then jumped to 7.13 at $K=4$ before falling again at $K=5-6$ as small fragments re-merge with the background -- while the mean TRUE coin count across these same 8 images is 5.4, confirming that $K=4$'s higher detected count reflects fragmentation rather than genuinely resolving more coins (its IoU actually falls to 0.298, well below $K=3$'s 0.529). The qualitative panel (coin_001.jpg, Figure 7) illustrates this directly: as K increases, individual coins increasingly fragment into separate colour clusters (highlight vs. shaded portions of the same coin), while a persistent thin sliver along a grout line is picked up as a spurious extra 'object' at every K tested -- a background artefact rather than a coin, and the same kind of texture-driven false positive discussed for coin_006.jpg in Section 6.6.

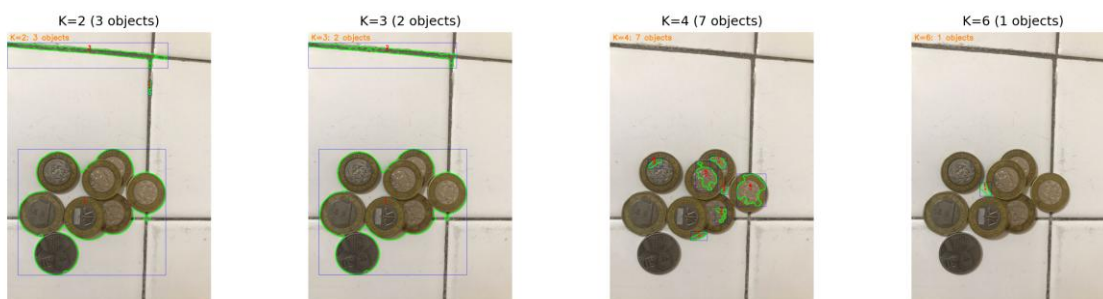
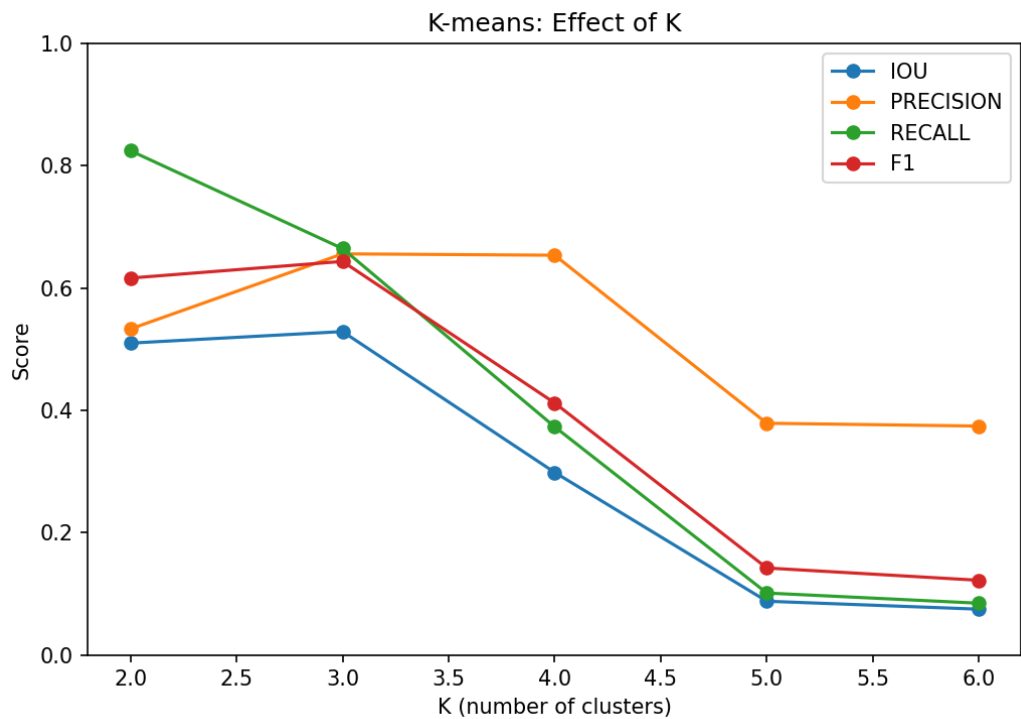


Figure 7: K-means output at K=2, 3, 4, 6 on coin_001.jpg.

6. Critical Analysis

This section addresses the seven critical-analysis questions posed in the assignment brief, using the quantitative results from Section 5.

6.1 Which segmentation method produced the best results, and why?

K-means (mean IoU = 0.528) achieved the best overall segmentation accuracy, narrowly ahead of thresholding (mean IoU = 0.508), with region growing clearly the weakest of the three (mean IoU = 0.298). This ordering matches the methods' underlying assumptions: thresholding's single global intensity cut works well whenever the coin/background brightness gap is large and fairly uniform across the frame (true for most bright and dim light-wood images), but has no way to recover once that assumption breaks (Sections 6.2, 6.6). K-means clusters on full Lab colour rather than grayscale intensity alone, which lets it separate coins from backgrounds of similar brightness but different hue -- its advantage over thresholding is modest on average but more consistent across conditions (Section 5.3). Region growing's weakness traces back to its dependence on automatically placed seeds (Section 6.3): whenever the bootstrap Otsu mask used to find seeds is itself poor, region growing has nothing to grow from at all, producing several near-zero-IoU images that pull its average down.

6.2 How did illumination changes affect thresholding?

Thresholding relies on Otsu's assumption that the image histogram is bimodal, with one mode for the coin and one for the background. Under dim or shadowed illumination, the intensity gap between coin and background narrows and can become non-bimodal (e.g. shadow pixels falling between the two peaks), which shifts or invalidates the automatically chosen threshold.

This was borne out in the results: for thresholding specifically, IoU fell from 0.791 (bright, light wood) to 0.266 (shadow, light wood) -- a drop of 0.53 -- and from 0.732 (bright, tile) to 0.193 (shadow, tile) -- a drop of 0.54 (eval_by_condition.csv). Both drops are of similar magnitude regardless of background material, confirming that it is the loss of a clean bimodal histogram under shadow, and not the specific background, that is responsible.

6.3 How did seed-point selection influence region-growing performance?

Because seeds are generated automatically from the distance-transform peaks of a rough Otsu mask, region growing's performance is coupled to that initial mask's quality: if the rough mask under- or over-segments (e.g. under poor illumination), seeds can be missed for some coins, or two touching coins can share a single distance-transform peak and therefore a single seed, causing them to be grown and reported as one merged object.

This was confirmed directly: the number of automatically placed seeds frequently fell well short of the true coin count in labels.csv, and not only for touching images. coin_001.jpg (8 coins, touching) produced only 3 seeds; coin_002.jpg (8 coins, touching, shadow) produced just 1; coin_018.jpg (6 coins, non-touching, shadow) produced only 3. The touching cases under-seed because several adjacent coins' distance-transform peaks merge into a single connected blob before the peak-finding step, so multiple coins collapse to one seed; the non-touching shadow case (coin_018) under-seeds for a different reason -- the bootstrap Otsu mask itself fails to separate several coins from the shadowed background at all, so no peak (and therefore no seed) is generated for them. Only coin_010.jpg (1 coin) and the well-lit, non-shadowed images produced seed counts reasonably close to the true count.

6.4 What effect did changing K have on K-means segmentation?

The IoU-vs-K curve (Section 5.5.2, `sweep_kmeans_k.csv`) peaks sharply at $K=3$ (IoU = 0.529), is close behind at $K=2$ (0.510), and then degrades steadily for $K>3$: $K=4$ falls to 0.298, $K=5$ to 0.088, and $K=6$ to 0.075. The qualitative panel (Figure 7) shows why: at $K=2-3$ the coins are captured as coherent regions, but from $K=4$ onward a single coin's surface increasingly splits across multiple colour clusters (its bright highlight vs. its shaded rim), fragmenting each coin into several small contours and inflating the mean detected object count (3.25 at $K=2$ to 7.13 at $K=4$) well past the mean true coin count (5.4), without any corresponding gain in overlap with the ground truth.

6.5 Which method was most computationally efficient?

Thresholding was by far the fastest method (mean 0.004 s/image), roughly 20x faster than region growing (0.073 s) and about 130x faster than K-means (0.476 s). This matches expectations: thresholding is a single global Otsu computation followed by one morphological pass; region growing performs a per-pixel flood-fill from several seeds, costing more but still a simple linear scan; K-means is the most expensive because it iteratively reassigns every pixel to one of K cluster centres, over multiple passes, across three colour channels.

6.6 Under what conditions did each method fail?

- **Thresholding:** Failed most severely on `coin_006.jpg` (brown wood, bright, non-touching), collapsing to IoU = 0.009: the wood grain's dark streaks were close enough in intensity to the coin edges that the polarity-selection heuristic locked onto the grain pattern rather than the coins, missing both coins almost entirely (Figure 5). It also degraded sharply under shadow generally (IoU dropping to 0.19-0.27, Section 6.2), since Otsu's bimodal assumption no longer holds once a shadow introduces a third intensity population.
- **Region growing:** Failed almost completely (IoU close to 0) on `coin_002.jpg` (tile, shadow, touching), `coin_018.jpg` (light wood, shadow) and `coin_006.jpg` (brown wood) -- in each case the bootstrap Otsu mask used to generate seeds was itself too poor to place a seed anywhere near the true coins, so growth either never started or grew into the wrong region entirely. On touching images that did produce seeds (e.g. `coin_001.jpg`), multiple physically distinct coins shared a single seed and were reported as one merged object instead of several separate ones.
- **K-means:** Also failed on `coin_006.jpg` (IoU = 0.0004): in Lab colour space, the bright, reflective faces of both coins were close enough to the pale wood background's lightness to be assigned to the same cluster as the background, leaving only the darker shadow rim around each coin's edge classified as foreground (Figure 6) -- the coin bodies themselves were almost entirely missed.

6.7 How could techniques studied in earlier weeks improve the segmentation results?

Several extensions from earlier in the course could plausibly improve on the current results, for example:

- The Watershed algorithm (built on the same distance-transform peaks already computed for region-growing seeds) to explicitly separate touching/overlapping coins that the current methods report as a single merged object.
- The Hough Circle Transform, exploiting the fact that coins are approximately circular, as a complementary detector for cases where intensity- or colour-based segmentation alone is ambiguous (e.g. low-contrast shadowed coins).

- Adaptive/local thresholding (e.g. Sauvola or Niblack) instead of a single global Otsu threshold, to better handle images with uneven illumination across the frame.

7. Discussion of Limitations and Failure Cases

All three methods are purely bottom-up (pixel intensity or colour based) and have no notion of “coin-ness” beyond local contrast against the background. Consequently:

- None of the three methods can reliably separate two touching coins into two objects, since there is no intensity or colour discontinuity at the point of contact — this is a structural limitation of intensity/colour-based segmentation, not a bug in a specific implementation.
- All three methods depend, directly or indirectly, on a reasonably bimodal separation between coin and background; a background with similar brightness/colour to the coins (e.g. certain wood tones under dim light) is the hardest case in this dataset.
- The automatic seed-generation step for region growing and the single global threshold in thresholding are both global, frame-level decisions, so a single image containing both a bright and a shadowed region can defeat either method even if most of the frame segments correctly.

The clearest evidence for this is `coin_006.jpg` (brown wood, bright lighting, 2 non-touching coins) -- nominally the 'easiest' image in the ground-truth subset by coin count, yet it produced the worst result of any image for all three methods: IoU = 0.009 (thresholding), 0.000 (region growing), and 0.0004 (K-means). All three failures trace back to the same root cause described in Section 6.6 -- the wood grain's texture and the coins' bright, reflective faces both have intensity/colour values that overlap heavily with the background, defeating every method that relies on coin/background contrast rather than shape. This is a strong illustration of a broader point: classical, bottom-up segmentation is fundamentally limited by how distinguishable the foreground is from the background in whatever feature space (intensity or colour) the method uses, regardless of which specific algorithm is applied on top.

8. Conclusion

Across this dataset, K-means ($K=3$) and Otsu thresholding are roughly tied as the best-performing methods (mean IoU 0.528 vs 0.508), each with a distinct advantage: thresholding is over two orders of magnitude faster and is the more practical choice when processing time matters or the background is fairly uniform, while K-means is more robust when the background varies in hue but not brightness (e.g. wood grain against pale coins). Region growing, despite being the most procedurally sophisticated of the three, was the weakest in practice because it inherits every weakness of the bootstrap Otsu mask used for seeding, and additionally loses touching coins to shared seeds.

The single biggest limitation demonstrated across all three methods is that none of them use shape information: a method aware that coins are approximately circular would not have been fooled by wood-grain streaks (coin_006.jpg) or lost touching coins to a shared seed (coin_001.jpg, coin_002.jpg). The most direct next step, and a natural extension of the pipeline already built here, would be to add a Watershed step on top of the existing distance-transform peaks (already computed for region-growing seeds) specifically to split touching coins into separate instances, combined with a Hough Circle check as a shape-based filter to reject non-circular false positives such as the grain-line detections seen in Section 6.6.

Appendix A: Complete Source Code

The complete, runnable source code for this project (label_masks.py, segmentation_pipeline.py, evaluate_segmentation.py, parameter_sweep.py) is submitted alongside this report and is also available at:

<https://github.com/Quojo-Steve/CoinIdentificationCV>

Evaluate_segmentation.py script

"""

evaluate_segmentation.py

Evaluates the three segmentation methods (thresholding, region growing, K-means) from segmentation_pipeline.py against your hand-labeled ground truth masks, and produces everything Question 4's "Experimental Requirement" asks for:

- Per-image / per-method IoU, Precision, Recall, F1
- Results broken down by condition (lighting, background, touching) using labels.csv
- The required method-comparison table:

Segmentation Accuracy	Processing Time	Sensitivity to Noise
Sensitivity to Illumination		
- Bar-chart visualisations of all of the above

Run this AFTER segmentation_pipeline.py (needs results/masks/<method>/ and results/segmentation_results.csv) and AFTER label_masks.py (needs masks/coin_XXX_mask.png ground truth for the labeled subset).

Usage:

pip install opencv-python numpy pandas matplotlib

```
python evaluate_segmentation.py
```

Run from the same folder as segmentation_pipeline.py (this script imports its segmentation functions directly so the sensitivity test uses the exact same algorithms).

Outputs:

```
results/eval_per_image.csv    <- IoU/P/R/F1 per labeled image/method
```

```
results/eval_by_condition.csv <- mean metrics grouped by condition
```

```
results/comparison_table.csv  <- the required 4-column summary table
```

```
results/plots/*.png          <- bar charts
```

```
"""
```

```
import os
```

```
import glob
```

```
import numpy as np
```

```
import pandas as pd
```

```
import cv2
```

```
import matplotlib
```

```
matplotlib.use("Agg")
```

```
import matplotlib.pyplot as plt
```

```
from segmentation_pipeline import (
```

```
    preprocess,
```

```
    segment_thresholding,
```

```
    segment_region_growing,
```

```
    segment_kmeans,
```

```
    METHODS,
```

)

```
GT_MASK_DIR = "masks"          # from label_masks.py
LABELS_CSV = "labels.csv"
PRED_MASK_DIR = "results/masks"  # from segmentation_pipeline.py
SEG_RESULTS_CSV = "results/segmentation_results.csv"
IMG_DIR = "dataset"
```

```
OUT_DIR = "results"
PLOT_DIR = os.path.join(OUT_DIR, "plots")
```

```
# -----
# Metrics
# -----
```

```
def compute_metrics(pred_mask, gt_mask):
    """Pixel-level IoU / Precision / Recall / F1 between two binary masks."""
    pred = (pred_mask > 0)
    gt = (gt_mask > 0)

    tp = np.logical_and(pred, gt).sum()
    fp = np.logical_and(pred, np.logical_not(gt)).sum()
    fn = np.logical_and(np.logical_not(pred), gt).sum()
    union = np.logical_or(pred, gt).sum()

    iou = tp / union if union > 0 else 0.0
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
```

```

recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
f1 = (2 * precision * recall / (precision + recall)
      if (precision + recall) > 0 else 0.0)
return iou, precision, recall, f1

```

```

def load_gt_masks():
    """filename -> ground truth binary mask, for every mask found in masks/."""
    gt = {}
    for path in glob.glob(os.path.join(GT_MASK_DIR, "*_mask.png")):
        base = os.path.basename(path).replace("_mask.png", ".jpg")
        m = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
        if m is not None:
            gt[base] = m
    return gt

```

```

# -----
# Part 1: evaluate existing predictions against ground truth
# -----

```

```

def evaluate_predictions(gt_masks, labels_df):
    rows = []
    for fname, gt_mask in gt_masks.items():
        for method in METHODS:
            pred_path = os.path.join(
                PRED_MASK_DIR, method, fname.replace(".jpg", "_mask.png")
            )

```

```

pred_mask = cv2.imread(pred_path, cv2.IMREAD_GRAYSCALE)

if pred_mask is None:

    print(f"Missing prediction: {pred_path} (run segmentation_pipeline.py first)")

    continue

if pred_mask.shape != gt_mask.shape:

    pred_mask = cv2.resize(

        pred_mask, (gt_mask.shape[1], gt_mask.shape[0]),

        interpolation=cv2.INTER_NEAREST

    )

iou, prec, rec, f1 = compute_metrics(pred_mask, gt_mask)

rows.append({

    "filename": fname, "method": method,

    "iou": iou, "precision": prec, "recall": rec, "f1": f1,

})

df = pd.DataFrame(rows)

if labels_df is not None and not df.empty:

    df = df.merge(labels_df, on="filename", how="left")

return df

```

```

# -----

# Part 2: noise / illumination sensitivity test

# -----

```

```

def add_gaussian_noise(img_bgr, sigma=25):

    noise = np.random.default_rng(0).normal(0, sigma, img_bgr.shape)

    noisy = np.clip(img_bgr.astype(np.float32) + noise, 0, 255).astype(np.uint8)

```

```
return noisy
```

```
def darken(img_bgr, factor=0.4):  
    return np.clip(img_bgr.astype(np.float32) * factor, 0, 255).astype(np.uint8)
```

```
def run_method_on_image(method, img_bgr):  
    gray, blurred = preprocess(img_bgr)  
    if method == "thresholding":  
        return segment_thresholding(blurred)  
    elif method == "region_growing":  
        return segment_region_growing(blurred)  
    elif method == "kmeans":  
        return segment_kmeans(img_bgr)  
    raise ValueError(method)
```

```
def sensitivity_test(gt_masks):  
    """  
    For every labeled image, re-run all three methods on (a) a noisy  
    version and (b) a darkened version, and compare IoU to the clean-image  
    IoU. The drop in IoU is the "sensitivity" figure for the comparison  
    table (bigger drop = more sensitive / less robust).  
    """  
    rows = []  
    for fname, gt_mask in gt_masks.items():  
        img_path = os.path.join(IMG_DIR, fname)
```



```
img_bgr = cv2.imread(img_path)
```

```
if img_bgr is None:
```

```
    print(f"Missing source image for sensitivity test: {img_path}")
```

```
    continue
```

```
variants = {
```

```
    "clean": img_bgr,
```

```
    "noisy": add_gaussian_noise(img_bgr),
```

```
    "dim": darken(img_bgr),
```

```
}
```

```
for method in METHODS:
```

```
    for variant_name, variant_img in variants.items():
```

```
        pred_mask = run_method_on_image(method, variant_img)
```

```
        iou, prec, rec, f1 = compute_metrics(pred_mask, gt_mask)
```

```
        rows.append({
```

```
            "filename": fname, "method": method,
```

```
            "variant": variant_name, "iou": iou,
```

```
        })
```

```
return pd.DataFrame(rows)
```

```
# -----
```

```
# Part 3: build the required comparison table
```

```
# -----
```

```
def build_comparison_table(eval_df, sens_df, seg_results_df):
```

```
    table_rows = []
```

for method in METHODS:

```
mean_iou = eval_df.loc[eval_df.method == method, "iou"].mean()
```

```
mean_time = np.nan
```

if seg_results_df is not None:

```
mean_time = seg_results_df.loc[
    seg_results_df.method == method, "processing_time_s"
].mean()
```

```
noise_drop = np.nan
```

```
illum_drop = np.nan
```

if not sens_df.empty:

```
pivot = sens_df[sens_df.method == method].pivot_table(
    index="filename", columns="variant", values="iou"
)
```

if {"clean", "noisy"}.issubset(pivot.columns):

```
noise_drop = (pivot["clean"] - pivot["noisy"]).mean()
```

if {"clean", "dim"}.issubset(pivot.columns):

```
illum_drop = (pivot["clean"] - pivot["dim"]).mean()
```

```
table_rows.append({
```

```
    "method": method,
```

```
    "segmentation_accuracy_iou": round(mean_iou, 4) if pd.notna(mean_iou) else None,
```

```
    "mean_processing_time_s": round(mean_time, 4) if pd.notna(mean_time) else None,
```

```
    "iou_drop_under_noise": round(noise_drop, 4) if pd.notna(noise_drop) else None,
```

```
    "iou_drop_under_dim_illumination": round(illum_drop, 4) if pd.notna(illum_drop) else None,
```

```
})
```

```
return pd.DataFrame(table_rows)
```

```

# -----

# Part 4: plots

# -----

def plot_metric_by_method(eval_df, metric, title, out_path):
    means = eval_df.groupby("method")[metric].mean().reindex(METHODS)
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.bar(means.index, means.values, color=["#4C72B0", "#DD8452", "#55A868"])
    ax.set_ylabel(metric.upper())
    ax.set_title(title)
    ax.set_ylim(0, 1)
    for i, v in enumerate(means.values):
        ax.text(i, v + 0.02, f"{v:.2f}", ha="center")
    plt.tight_layout()
    plt.savefig(out_path, dpi=150)
    plt.close(fig)

def plot_metric_by_condition(eval_df, condition_col, metric, title, out_path):
    if condition_col not in eval_df.columns:
        return
    pivot = eval_df.pivot_table(
        index=condition_col, columns="method", values=metric, aggfunc="mean"
    ).reindex(columns=METHODS)
    fig, ax = plt.subplots(figsize=(7, 4))
    pivot.plot(kind="bar", ax=ax)

```

```
ax.set_ylabel(metric.upper())
ax.set_title(title)
ax.set_ylim(0, 1)
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.savefig(out_path, dpi=150)
plt.close(fig)
```

```
def plot_processing_time(seg_results_df, out_path):
    if seg_results_df is None:
        return

    means = seg_results_df.groupby("method")["processing_time_s"].mean().reindex(METHODS)
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.bar(means.index, means.values, color=["#4C72B0", "#DD8452", "#55A868"])
    ax.set_ylabel("Mean processing time (s)")
    ax.set_title("Processing Time by Method")
    for i, v in enumerate(means.values):
        ax.text(i, v, f"{v:.3f}s", ha="center", va="bottom")
    plt.tight_layout()
    plt.savefig(out_path, dpi=150)
    plt.close(fig)

def plot_sensitivity(sens_df, out_path):
    if sens_df.empty:
        return

    pivot = sens_df.pivot_table(index="method", columns="variant", values="iou", aggfunc="mean")
```

```

pivot = pivot.reindex(index=METHODS, columns=["clean", "noisy", "dim"])

fig, ax = plt.subplots(figsize=(7, 4))

pivot.plot(kind="bar", ax=ax)

ax.set_ylabel("Mean IoU")

ax.set_title("Robustness to Noise / Dim Illumination")

ax.set_ylim(0, 1)

plt.xticks(rotation=0)

plt.tight_layout()

plt.savefig(out_path, dpi=150)

plt.close(fig)

```

```

# -----
# Main
# -----

```

```

def main():

    os.makedirs(PLOT_DIR, exist_ok=True)

    gt_masks = load_gt_masks()

    if not gt_masks:

        print(f"No ground truth masks found in {GT_MASK_DIR}/. "
              f"Run label_masks.py first.")

        return

    print(f"Loaded {len(gt_masks)} ground-truth masks.")

    labels_df = pd.read_csv(LABELS_CSV) if os.path.exists(LABELS_CSV) else None

    seg_results_df = (pd.read_csv(SEG_RESULTS_CSV)

```

```

        if os.path.exists(SEG_RESULTS_CSV) else None)

# Part 1: per-image evaluation against ground truth
eval_df = evaluate_predictions(gt_masks, labels_df)
eval_df.to_csv(os.path.join(OUT_DIR, "eval_per_image.csv"), index=False)
print("Saved results/eval_per_image.csv")

# Grouped by condition, if labels.csv had condition columns
condition_cols = [c for c in ["lighting_condition", "background", "touching_objects(y/n)"]
                   if labels_df is not None and c in eval_df.columns]
if condition_cols:
    by_cond = eval_df.groupby(condition_cols + ["method"])[
        ["iou", "precision", "recall", "f1"]
    ].mean().reset_index()
    by_cond.to_csv(os.path.join(OUT_DIR, "eval_by_condition.csv"), index=False)
    print("Saved results/eval_by_condition.csv")

# Part 2: sensitivity test (synthetic noise + dim variants)
print("Running noise/illumination sensitivity test "
      "(re-segmenting noisy + darkened versions of each labeled image)...")
sens_df = sensitivity_test(gt_masks)
sens_df.to_csv(os.path.join(OUT_DIR, "sensitivity_results.csv"), index=False)
print("Saved results/sensitivity_results.csv")

# Part 3: comparison table
comparison_df = build_comparison_table(eval_df, sens_df, seg_results_df)
comparison_df.to_csv(os.path.join(OUT_DIR, "comparison_table.csv"), index=False)
print("Saved results/comparison_table.csv")

```

```
print(comparison_df.to_string(index=False))
```

```
# Part 4: plots
```

```
plot_metric_by_method(eval_df, "iou", "Mean IoU by Method",  
                       os.path.join(PLOT_DIR, "iou_by_method.png"))  
plot_metric_by_method(eval_df, "precision", "Mean Precision by Method",  
                       os.path.join(PLOT_DIR, "precision_by_method.png"))  
plot_metric_by_method(eval_df, "recall", "Mean Recall by Method",  
                       os.path.join(PLOT_DIR, "recall_by_method.png"))  
plot_metric_by_method(eval_df, "f1", "Mean F1 by Method",  
                       os.path.join(PLOT_DIR, "f1_by_method.png"))
```

```
if "lighting_condition" in eval_df.columns:
```

```
    plot_metric_by_condition(  
        eval_df, "lighting_condition", "iou",  
        "IoU by Lighting Condition and Method",  
        os.path.join(PLOT_DIR, "iou_by_lighting.png")  
    )
```

```
if "touching_objects(y/n)" in eval_df.columns:
```

```
    plot_metric_by_condition(  
        eval_df, "touching_objects(y/n)", "iou",  
        "IoU by Touching vs Non-Touching Coins",  
        os.path.join(PLOT_DIR, "iou_by_touching.png")  
    )
```

```
plot_processing_time(seg_results_df, os.path.join(PLOT_DIR, "processing_time.png"))
```

```
plot_sensitivity(sens_df, os.path.join(PLOT_DIR, "sensitivity.png"))
```

```
print(f"\nAll plots saved to {PLOT_DIR}/")
```

```
if __name__ == "__main__":  
    main()
```

Label_masks.py script

```
"""
```

Ground-truth mask labeling tool for the coin segmentation project.

```
pip install opencv-python matplotlib numpy
```

```
python label_masks.py
```

Run this from the folder that contains your 'dataset/' folder

(the one with coin_001.jpg ... coin_023.jpg), or edit IMG_DIR below.

HOW TO USE (per coin, per image):

- Left-click points around one coin's edge (roughly following its boundary, 6-10 points is plenty for a coin).
- Press ENTER (or middle-click) to close that coin's outline.
- Right-click to undo the last point if you misclick.
- After closing a coin's outline, the window updates showing your progress in green. Keep clicking to outline the next coin.
- When you've outlined every coin in the image, just CLOSE the window (the X button) to move to the next image.

The mask for each image is saved as a binary PNG (white = coin, black = background) in the 'masks/' folder, ready for IoU / Precision / Recall later.


```

"""

import os

import numpy as np

import cv2

import matplotlib.pyplot as plt

IMG_DIR = "dataset"

MASK_DIR = "masks"

# Representative subset spanning lighting, background, count, and touching conditions.
# Edit this list if you'd rather label a different set of images.
SELECTED = [

    "coin_001.jpg", # 8 coins, tile, bright, touching
    "coin_002.jpg", # 8 coins, tile, shadow, touching
    "coin_006.jpg", # 2 coins, brown wood, bright, not touching (easy case)
    "coin_010.jpg", # 1 coin, light wood, dim (easiest case)
    "coin_012.jpg", # 6 coins, light wood, blurry, touching
    "coin_017.jpg", # 6 coins, light wood, bright, not touching
    "coin_018.jpg", # 6 coins, light wood, shadow, not touching
    "coin_022.jpg", # 6 coins, light wood, dim, not touching
]

def label_image(path):

    img_bgr = cv2.imread(path)

    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

    h, w = img_bgr.shape[:2]

    mask = np.zeros((h, w), dtype=np.uint8)

    coin_count = 0

    fig, ax = plt.subplots(figsize=(8, 8 * h / w))

    ax.imshow(img_rgb)

```

```

ax.set_title(
    "Click around a coin's edge, ENTER to close it.\n"
    "Right-click = undo last point. Close window when all coins are done."
)

plt.tight_layout()
fig.canvas.draw()

while True:
    pts = plt.ginput(n=-1, timeout=0)
    if plt.fignum_exists(fig.number) is False:
        break # window was closed
    if len(pts) < 3:
        continue # not enough points for a polygon, try again
    poly = np.array(pts, dtype=np.int32)
    cv2.fillPoly(mask, [poly], 255)
    coin_count += 1

    overlay = img_rgb.copy()
    overlay[mask > 0] = (0, 255, 0)
    blended = cv2.addWeighted(img_rgb, 0.6, overlay, 0.4, 0)
    ax.clear()
    ax.imshow(blended)
    ax.set_title(
        f"{coin_count} coin(s) outlined so far.\n"
        "Click next coin, or close window when finished."
    )
    fig.canvas.draw()

```

```
plt.close(fig)

return mask, coin_count
```

```
def main():

    os.makedirs(MASK_DIR, exist_ok=True)

    for fname in SELECTED:

        path = os.path.join(IMG_DIR, fname)

        if not os.path.exists(path):

            print(f"Skipping missing file: {path}")

            continue

        print(f"\nLabeling {fname} -- outline each coin, close window when done.")

        mask, n = label_image(path)

        out_path = os.path.join(MASK_DIR, fname.replace(".jpg", "_mask.png"))

        cv2.imwrite(out_path, mask)

        print(f"Saved mask with {n} coin(s) -> {out_path}")

    print("\nAll done. Ground-truth masks saved in:", MASK_DIR)
```

```
if __name__ == "__main__":

    main()
```

Segmentation_pipeline.py script

```
"""
```

```
segmentation_pipeline.py
```

Implements three classical segmentation approaches for the coin

counting/measurement task (CSCD608 - Question 4):

1. Thresholding (Otsu global threshold)
2. Region growing (seeded, intensity-similarity based)
3. K-means clustering (Lab colour space, K configurable)

For each image + method, this script:

- preprocesses the image (grayscale, denoise)
- produces a binary foreground mask
- extracts per-object measurements via contours
(count, area, perimeter, centroid, bounding box, enclosing circle)
- saves a visualisation (contours + bboxes + numbered coins)
- saves the binary mask (needed later for IoU against your ground truth)
- records processing time per image/method

Usage:

```
pip install opencv-python numpy pandas
```

```
python segmentation_pipeline.py
```

Run this from the folder that contains 'dataset/' (coin_001.jpg ... coin_023.jpg).

Outputs:

```
results/masks/<method>/coin_XXX_mask.png
```

```
results/viz/<method>/coin_XXX_viz.png
```

```
results/segmentation_results.csv  <- one row per (image, method)
```

```
"""
```

```
import os
```

```
import time
```

```

import glob

import numpy as np

import pandas as pd

import cv2

# -----

# Config
# -----

IMG_DIR = "dataset"

OUT_DIR = "results"

MIN_OBJECT_AREA = 300    # filter out tiny noise contours (pixels^2)

KMEANS_K = 3            # default K; try 2, 3, 4 for your K-sweep experiment

REGION_GROW_THRESH = 12  # intensity similarity threshold for region growing

METHODS = ["thresholding", "region_growing", "kmeans"]

# -----

# Preprocessing
# -----

def preprocess(img_bgr):

    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

    blurred = cv2.GaussianBlur(gray, (5, 5), 0)

    return gray, blurred

```

```

# -----
# Method 1: Thresholding (Otsu + morphology)
# -----

def otsu_best_polarity(blurred_gray, target_fraction=0.25):
    """
    Coins can be darker or lighter than the background depending on the
    photo (tile vs. brown wood vs. light wood), so compute both Otsu
    polarities and keep whichever gives a more plausible "coins are a
    minority of the frame" foreground. Shared by thresholding and the
    region-growing seed step so both agree on which side is "coin".
    """
    _, mask_inv = cv2.threshold(
        blurred_gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
    )
    _, mask_norm = cv2.threshold(
        blurred_gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )

    def fg_fraction(m):
        return np.count_nonzero(m) / m.size

    candidates = [mask_inv, mask_norm]
    return min(candidates, key=lambda m: abs(fg_fraction(m) - target_fraction))

def segment_thresholding(blurred_gray):
    mask = otsu_best_polarity(blurred_gray)

```

```

kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel, iterations=2)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel, iterations=2)
return mask

```

```

# -----

```

```

# Method 2: Region growing (auto-seeded)

```

```

# -----

```

```

def get_seed_points(mask_hint):

```

```

    """

```

```

    Find one seed point per likely coin using the distance transform of a
    rough mask (peaks of the distance transform sit near coin centers).

```

```

    This avoids requiring manual seed clicks for all 23 images.

```

```

    """

```

```

    dist = cv2.distanceTransform(mask_hint, cv2.DIST_L2, 5)

```

```

    if dist.max() == 0:

```

```

        return []

```

```

    _, peaks = cv2.threshold(dist, 0.5 * dist.max(), 255, 0)

```

```

    peaks = peaks.astype(np.uint8)

```

```

    n_labels, labels = cv2.connectedComponents(peaks)

```

```

    seeds = []

```

```

    for lbl in range(1, n_labels):

```

```

        ys, xs = np.where(labels == lbl)

```

```

        if len(xs) == 0:

```

```

            continue

```

```

        seeds.append((int(np.mean(ys)), int(np.mean(xs))))

```

```
return seeds
```

```
def region_growing(gray, seeds, thresh=REGION_GROW_THRESH):
```

```
    h, w = gray.shape
```

```
    mask = np.zeros((h, w), dtype=np.uint8)
```

```
    visited = np.zeros((h, w), dtype=bool)
```

```
    for (sy, sx) in seeds:
```

```
        if visited[sy, sx]:
```

```
            continue
```

```
        seed_val = int(gray[sy, sx])
```

```
        stack = [(sy, sx)]
```

```
        visited[sy, sx] = True
```

```
        while stack:
```

```
            y, x = stack.pop()
```

```
            mask[y, x] = 255
```

```
            for dy, dx in ((-1, 0), (1, 0), (0, -1), (0, 1)):
```

```
                ny, nx = y + dy, x + dx
```

```
                if 0 <= ny < h and 0 <= nx < w and not visited[ny, nx]:
```

```
                    if abs(int(gray[ny, nx]) - seed_val) <= thresh:
```

```
                        visited[ny, nx] = True
```

```
                        stack.append((ny, nx))
```

```
    return mask
```

```
def segment_region_growing(blurred_gray, thresh=REGION_GROW_THRESH):
```

```
    # Bootstrap seed locations from a quick Otsu pass (used only to find
```



```
# plausible coin centers -- the actual mask comes from growing).  
# Uses the same polarity-selection as thresholding, since coins are  
# darker than the background in some photos and lighter in others.  
rough = otsu_best_polarity(blurred_gray)  
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))  
rough = cv2.morphologyEx(rough, cv2.MORPH_OPEN, kernel, iterations=2)
```

```
seeds = get_seed_points(rough)  
if not seeds:  
    return np.zeros_like(blurred_gray)
```

```
mask = region_growing(blurred_gray, seeds, thresh=thresh)  
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel, iterations=2)  
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel, iterations=1)  
return mask
```

```
# -----  
# Method 3: K-means clustering  
# -----
```

```
def segment_kmeans(img_bgr, k=KMEANS_K):  
    lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)  
    h, w = lab.shape[:2]  
    samples = lab.reshape((-1, 3)).astype(np.float32)  
  
    criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.5)  
    _, labels, _ = cv2.kmeans(
```

```
    samples, k, None, criteria, attempts=5, flags=cv2.KMEANS_PP_CENTERS
)
labels = labels.reshape((h, w))
```

```
# Heuristic: the background is usually the largest cluster by pixel
# count. Treat everything else as foreground (coins + shadow edges).
counts = np.bincount(labels.flatten(), minlength=k)
bg_cluster = int(np.argmax(counts))
remaining = [c for c in range(k) if c != bg_cluster]
coin_cluster = min(remaining, key=lambda c: counts[c])
mask = np.where(labels == coin_cluster, 255, 0).astype(np.uint8)

kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel, iterations=2)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel, iterations=2)
return mask
```

```
# -----
# Object measurement (shared across all three methods)
# -----
```

```
def measure_objects(mask, min_area=MIN_OBJECT_AREA):
    contours, _ = cv2.findContours(
        mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )
    objects = []
    for cnt in contours:
```

```

area = cv2.contourArea(cnt)

if area < min_area:

    continue

perimeter = cv2.arcLength(cnt, True)

M = cv2.moments(cnt)

if M["m00"] == 0:

    continue

cx, cy = M["m10"] / M["m00"], M["m01"] / M["m00"]

x, y, w, h = cv2.boundingRect(cnt)

(ecx, ecy), radius = cv2.minEnclosingCircle(cnt)

objects.append({

    "area": area,

    "perimeter": perimeter,

    "centroid": (cx, cy),

    "bbox": (x, y, w, h),

    "enclosing_circle": ((ecx, ecy), radius),

    "contour": cnt,

})

return objects

```

```

def draw_overlay(img_bgr, objects, method_name):

    vis = img_bgr.copy()

    for i, obj in enumerate(objects, start=1):

        cv2.drawContours(vis, [obj["contour"]], -1, (0, 255, 0), 2)

        x, y, w, h = obj["bbox"]

        cv2.rectangle(vis, (x, y), (x + w, y + h), (255, 0, 0), 1)

        cx, cy = obj["centroid"]

```

```

cv2.circle(vis, (int(cx), int(cy)), 3, (0, 0, 255), -1)

cv2.putText(
    vis, str(i), (int(cx) - 8, int(cy) - 8),
    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2
)
cv2.putText(
    vis, f"{method_name}: {len(objects)} objects", (10, 30),
    cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 128, 255), 2
)
return vis

```

```

# -----

```

```

# Main pipeline

```

```

# -----

```

```

def run_method(method, img_bgr, gray, blurred, region_grow_thresh=REGION_GROW_THRESH,
k=KMEANS_K):

```

```

    start = time.time()

```

```

    if method == "thresholding":

```

```

        mask = segment_thresholding(blurred)

```

```

    elif method == "region_growing":

```

```

        mask = segment_region_growing(blurred, thresh=region_grow_thresh)

```

```

    elif method == "kmeans":

```

```

        mask = segment_kmeans(img_bgr, k=k)

```

```

    else:

```

```

        raise ValueError(method)

```

```

    elapsed = time.time() - start

```

```
return mask, elapsed
```

```
def main():
```

```
    for method in METHODS:
```

```
        os.makedirs(os.path.join(OUT_DIR, "masks", method), exist_ok=True)
```

```
        os.makedirs(os.path.join(OUT_DIR, "viz", method), exist_ok=True)
```

```
image_paths = sorted(glob.glob(os.path.join(IMG_DIR, "coin_*.jpg")))
```

```
if not image_paths:
```

```
    print(f"No images found in {IMG_DIR}/. Check IMG_DIR path.")
```

```
    return
```

```
rows = []
```

```
for path in image_paths:
```

```
    fname = os.path.basename(path)
```

```
    img_bgr = cv2.imread(path)
```

```
    if img_bgr is None:
```

```
        print(f"Could not read {path}, skipping.")
```

```
        continue
```

```
    gray, blurred = preprocess(img_bgr)
```

```
    print(f"Processing {fname} ...")
```

```
    for method in METHODS:
```

```
        mask, elapsed = run_method(method, img_bgr, gray, blurred)
```

```
        objects = measure_objects(mask)
```

```
        vis = draw_overlay(img_bgr, objects, method)
```

```
mask_path = os.path.join(
    OUT_DIR, "masks", method, fname.replace(".jpg", "_mask.png")
)
```

```
viz_path = os.path.join(
    OUT_DIR, "viz", method, fname.replace(".jpg", "_viz.png")
)
```

```
cv2.imwrite(mask_path, mask)
```

```
cv2.imwrite(viz_path, vis)
```

```
total_area = sum(o["area"] for o in objects)
```

```
total_perimeter = sum(o["perimeter"] for o in objects)
```

```
rows.append({
    "filename": fname,
    "method": method,
    "num_detected": len(objects),
    "total_area_px": total_area,
    "total_perimeter_px": total_perimeter,
    "processing_time_s": round(elapsed, 4),
    "mask_path": mask_path,
    "viz_path": viz_path,
})
```

```
df = pd.DataFrame(rows)
```

```
csv_path = os.path.join(OUT_DIR, "segmentation_results.csv")
```

```
df.to_csv(csv_path, index=False)
```

```
print(f"\nDone. Results written to {csv_path}")
```

```
print(f"Masks in {OUT_DIR}/masks/<method>/, visualisations in {OUT_DIR}/viz/<method>/")
```

```
if __name__ == "__main__":  
    main()
```

Parameter_sweep.py script

```
"""
```

```
parameter_sweep.py
```

Investigates how the tunable parameters of two segmentation methods affect results, as required by Question 4's tasks:

- "Implement region-growing segmentation and investigate the effect of seed-point selection and similarity criteria."
- "Implement K-means clustering ... and investigate the effect of different values of K."

For region growing, sweeps the intensity-similarity threshold used when growing from each seed (a tighter threshold = stricter similarity criterion = smaller/fragmented regions; a looser threshold = regions bleed into background/shadows). Seed *selection* itself already comes from an automatic distance-transform step in `segmentation_pipeline.py`; this script reports how many seeds that step finds per image, since seed count/placement is what "seed-point selection" drives.

For K-means, sweeps K itself.

For each parameter value, this script re-segments every LABELED image (the ones with ground-truth masks in `masks/`) and records IoU, precision, recall, F1, detected object count, and processing time -- so you can

show both segmentation quality and object-count accuracy as a function of the parameter.

Run this AFTER label_masks.py (needs masks/coin_XXX_mask.png).

Usage:

```
python parameter_sweep.py
```

Outputs:

```
results/sweep_region_growing_threshold.csv
```

```
results/sweep_kmeans_k.csv
```

```
results/plots/sweep_region_growing_threshold.png
```

```
results/plots/sweep_kmeans_k.png
```

```
results/plots/sweep_kmeans_k_qualitative_<image>.png (visual K comparison)
```

```
"""
```

```
import os
```

```
import glob
```

```
import time
```

```
import numpy as np
```

```
import pandas as pd
```

```
import cv2
```

```
import matplotlib
```

```
matplotlib.use("Agg")
```

```
import matplotlib.pyplot as plt
```

```
from segmentation_pipeline import (
```

```
    preprocess,
```



```
    segment_region_growing,  
    segment_kmeans,  
    get_seed_points,  
    otsu_best_polarity,  
    measure_objects,  
    draw_overlay,  
)  
from evaluate_segmentation import compute_metrics, load_gt_masks
```

```
IMG_DIR = "dataset"
```

```
OUT_DIR = "results"
```

```
PLOT_DIR = os.path.join(OUT_DIR, "plots")
```

```
REGION_GROW_THRESHOLDS = [5, 10, 15, 20, 25, 30]
```

```
KMEANS_K_VALUES = [2, 3, 4, 5, 6]
```

```
# -----
```

```
# Region growing threshold sweep
```

```
# -----
```

```
def sweep_region_growing(gt_masks):  
    rows = []  
    for fname, gt_mask in gt_masks.items():  
        img_path = os.path.join(IMG_DIR, fname)  
        img_bgr = cv2.imread(img_path)  
        if img_bgr is None:  
            continue
```

```

gray, blurred = preprocess(img_bgr)

# Report seed count once per image (seed *selection* is automatic;
# this shows how many candidate coin centers it finds).

rough = otsu_best_polarity(blurred)

kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))

rough = cv2.morphologyEx(rough, cv2.MORPH_OPEN, kernel, iterations=2)

n_seeds = len(get_seed_points(rough))

for thresh in REGION_GROW_THRESHOLDS:

    start = time.time()

    mask = segment_region_growing(blurred, thresh=thresh)

    elapsed = time.time() - start

    iou, prec, rec, f1 = compute_metrics(mask, gt_mask)

    objects = measure_objects(mask)

    rows.append({

        "filename": fname, "threshold": thresh, "n_seeds": n_seeds,

        "num_detected": len(objects), "iou": iou,

        "precision": prec, "recall": rec, "f1": f1,

        "processing_time_s": round(elapsed, 4),

    })

return pd.DataFrame(rows)

# -----

# K-means K sweep

# -----

def sweep_kmeans(gt_masks):

```

```

rows = []

for fname, gt_mask in gt_masks.items():

    img_path = os.path.join(IMG_DIR, fname)

    img_bgr = cv2.imread(img_path)

    if img_bgr is None:
        continue

    for k in KMEANS_K_VALUES:

        start = time.time()

        mask = segment_kmeans(img_bgr, k=k)

        elapsed = time.time() - start

        iou, prec, rec, f1 = compute_metrics(mask, gt_mask)

        objects = measure_objects(mask)

        rows.append({

            "filename": fname, "k": k,

            "num_detected": len(objects), "iou": iou,

            "precision": prec, "recall": rec, "f1": f1,

            "processing_time_s": round(elapsed, 4),

        })

    return pd.DataFrame(rows)

def save_kmeans_qualitative_examples(gt_masks, k_values=(2, 3, 4, 6), max_images=2):

    """Side-by-side visual comparison of K-means output at different K,

    for a couple of representative images -- useful evidence in the report."""

    for fname in list(gt_masks.keys())[:max_images]:

        img_path = os.path.join(IMG_DIR, fname)

        img_bgr = cv2.imread(img_path)

        if img_bgr is None:

```

```
continue
```

```
fig, axes = plt.subplots(1, len(k_values), figsize=(4 * len(k_values), 4))
for ax, k in zip(axes, k_values):
    mask = segment_kmeans(img_bgr, k=k)
    objects = measure_objects(mask)
    vis = draw_overlay(img_bgr, objects, f"K={k}")
    vis_rgb = cv2.cvtColor(vis, cv2.COLOR_BGR2RGB)
    ax.imshow(vis_rgb)
    ax.set_title(f"K={k} ({len(objects)} objects)")
    ax.axis("off")
plt.tight_layout()
out_path = os.path.join(
    PLOT_DIR, f"sweep_kmeans_k_qualitative_{fname.replace('.jpg', '')}.png"
)
plt.savefig(out_path, dpi=150)
plt.close(fig)
print(f"Saved {out_path}")
```

```
# -----
```

```
# Plots
```

```
# -----
```

```
def plot_region_growing_sweep(df, out_path):
    means = df.groupby("threshold")[["iou", "precision", "recall", "f1"]].mean()
    fig, ax1 = plt.subplots(figsize=(7, 5))
    for col in ["iou", "precision", "recall", "f1"]:
        ax1.plot(means.index, means[col], marker="o", label=col.upper())
    ax1.set_xlabel("Region-growing similarity threshold")
```

```

ax1.set_ylabel("Score")

ax1.set_ylim(0, 1)

ax1.set_title("Region Growing: Effect of Similarity Threshold")

ax1.legend()

plt.tight_layout()

plt.savefig(out_path, dpi=150)

plt.close(fig)

```

```

def plot_kmeans_sweep(df, out_path):

    means = df.groupby("k")[["iou", "precision", "recall", "f1"]].mean()

    fig, ax1 = plt.subplots(figsize=(7, 5))

    for col in ["iou", "precision", "recall", "f1"]:

        ax1.plot(means.index, means[col], marker="o", label=col.upper())

    ax1.set_xlabel("K (number of clusters)")

    ax1.set_ylabel("Score")

    ax1.set_ylim(0, 1)

    ax1.set_title("K-means: Effect of K")

    ax1.legend()

    plt.tight_layout()

    plt.savefig(out_path, dpi=150)

    plt.close(fig)

```

```

# -----

```

```

# Main

```

```

# -----

```

```

def main():

    os.makedirs(PLOT_DIR, exist_ok=True)

    gt_masks = load_gt_masks()

    if not gt_masks:

```

```

    print("No ground-truth masks found in masks/. Run label_masks.py first.")

    return

print(f"Loaded {len(gt_masks)} ground-truth masks. Running sweeps...")

print("\nSweeping region-growing similarity threshold:", REGION_GROW_THRESHOLDS)

rg_df = sweep_region_growing(gt_masks)

rg_df.to_csv(os.path.join(OUT_DIR, "sweep_region_growing_threshold.csv"), index=False)

print(rg_df.groupby("threshold")[["iou", "num_detected"]].mean())

plot_region_growing_sweep(rg_df, os.path.join(PLOT_DIR, "sweep_region_growing_threshold.png"))

print("\nSweeping K-means K:", KMEANS_K_VALUES)

km_df = sweep_kmeans(gt_masks)

km_df.to_csv(os.path.join(OUT_DIR, "sweep_kmeans_k.csv"), index=False)

print(km_df.groupby("k")[["iou", "num_detected"]].mean())

plot_kmeans_sweep(km_df, os.path.join(PLOT_DIR, "sweep_kmeans_k.png"))

print("\nSaving qualitative K-means comparison images...")

save_kmeans_qualitative_examples(gt_masks)

print(f"\nDone. CSVs in {OUT_DIR}/, plots in {PLOT_DIR}/")

if __name__ == "__main__":
    main()

```